

Applied Cryptography Program 2026

Week4: 「証明」と「ゼロ知識性」

小林 聖弥 | Seiya Kobayashi

目次

レクチャー

1. ゼロ知識証明の全体像 (Recap)
2. 証明とはなにか
3. さまざまな証明スキーム
4. ゼロ知識性の実現手法
5. まとめ

ワークショップ

～ZK, or not ZK～

ケーススタディ: AIエージェント保険

目次

レクチャー

1. ゼロ知識証明の全体像 (Recap)
2. 証明とはなにか
3. さまざまな証明スキーム
4. ゼロ知識性の実現手法
5. まとめ

ワークショップ

～ZK, or not ZK～

ケーススタディ: AIエージェント保険

署名と証明

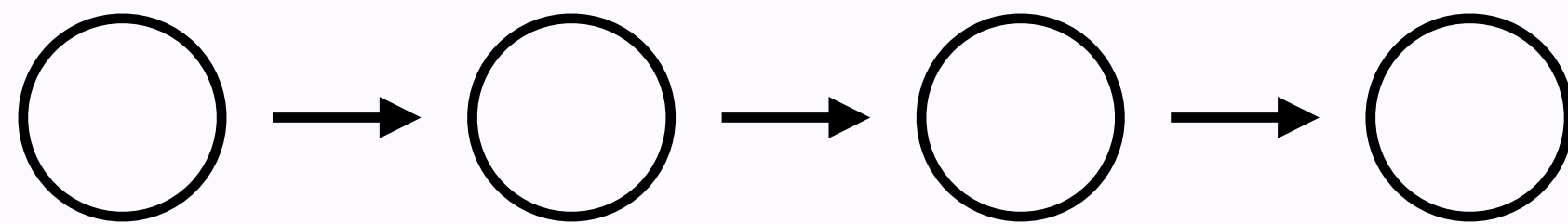
Q. 電子署名アルゴリズムとゼロ知識証明の違いは？

ゼロ知識証明

コミットメントスキーム (CS)

全体フロー

1. SetUp 2. Commit 3. Open 4. Verify



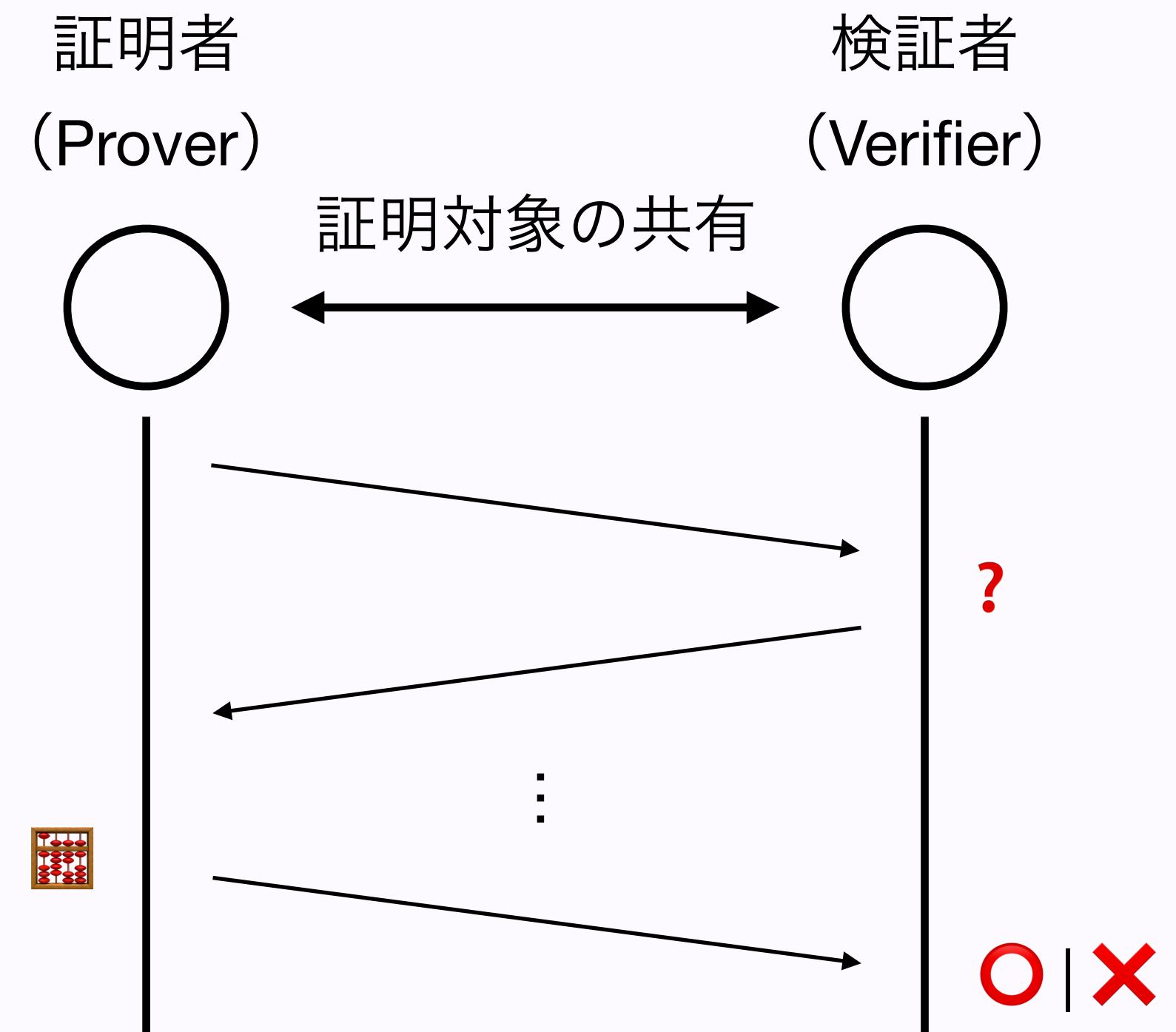
満たすべき性質

1. 拘束・束縛性 (binding)
2. 秘匿・隠蔽性 (hiding)

Q: ハッシュ関数 vs. ハッシュコミットメント？

1. ゼロ知識証明の全体像 (Recap)

証明システム (PS)



Q: 対話型と非対話型はどのように使い分ける？

目次

レクチャー

1. ゼロ知識証明の全体像 (Recap)
2. 証明とはなにか
3. さまざまな証明スキーム
4. ゼロ知識性の実現手法
5. まとめ

ワークショップ

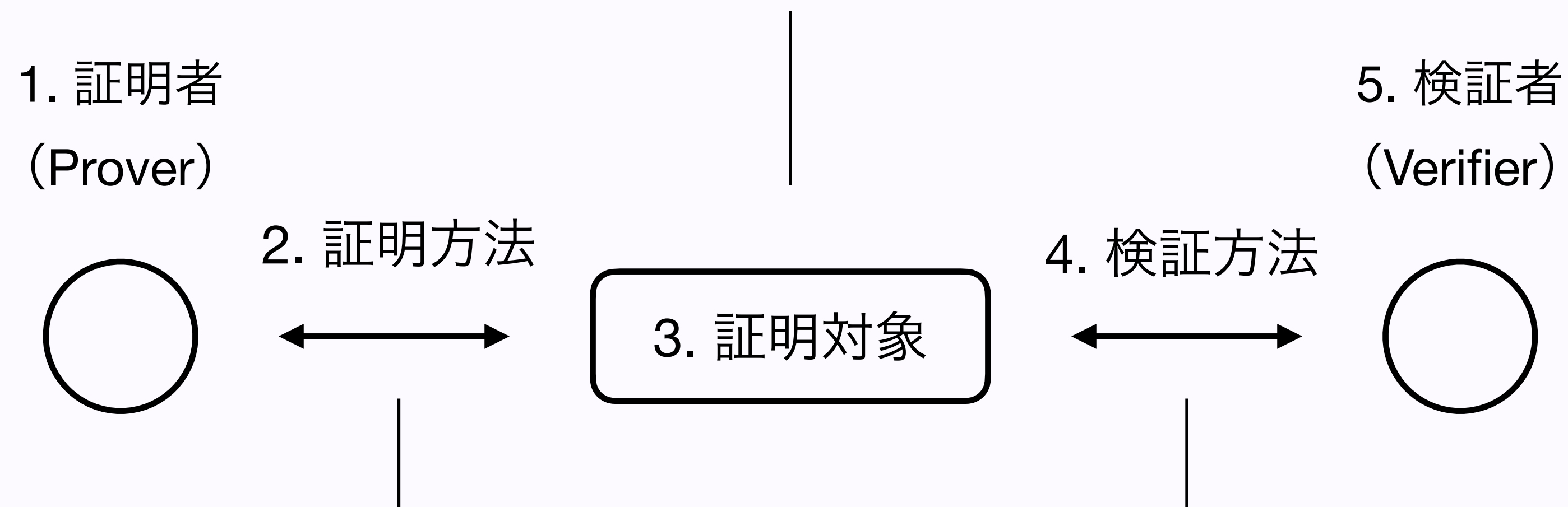
～ZK, or not ZK～

ケーススタディ: AIエージェント保険

概念としての証明

証明という概念は、**5つのパート**から成立する

Q. 我々は日常的に何を証明していますか？証明する対象に共通項はありますか？

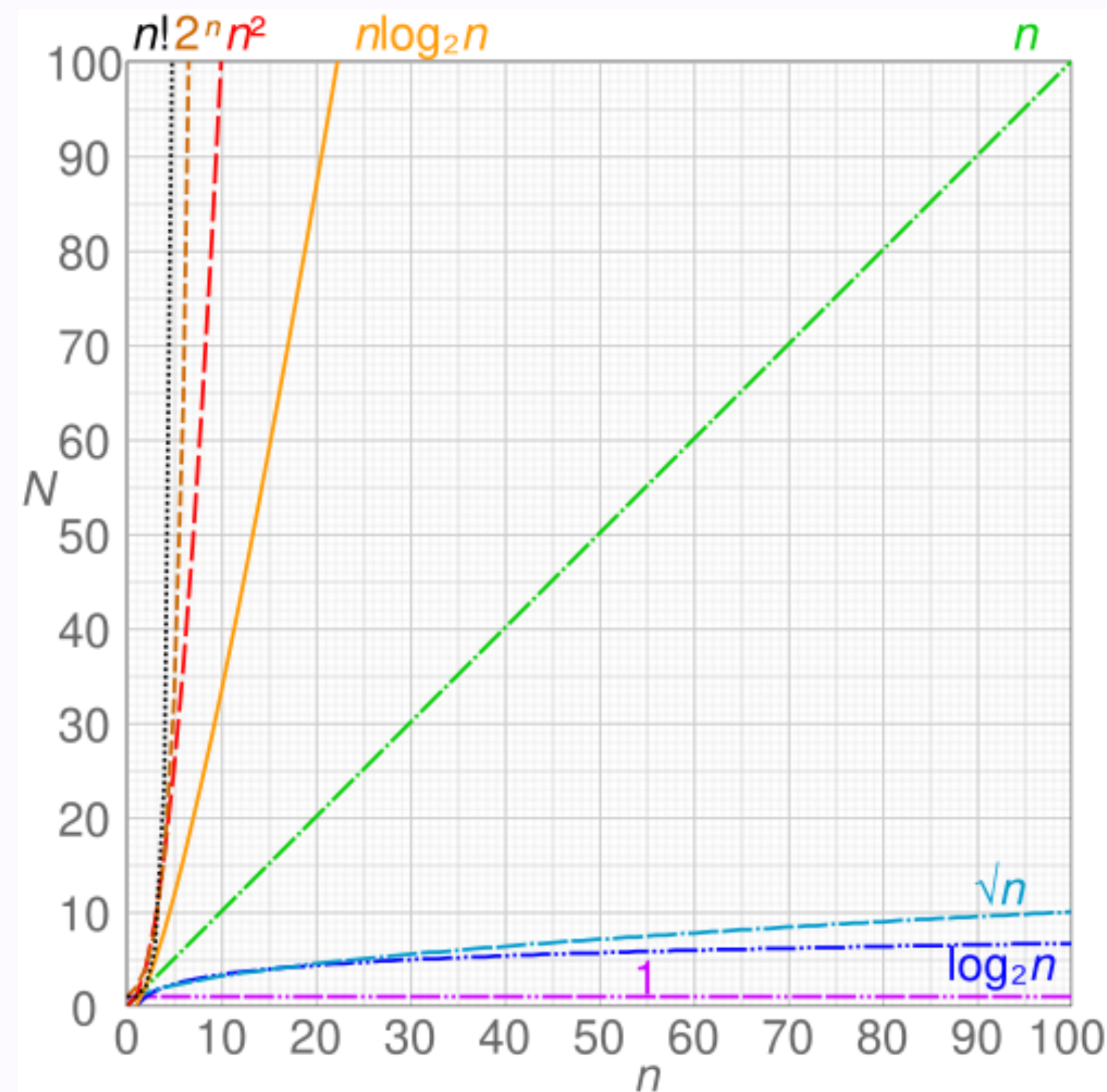


Q. 物事の証明・検証方法にはどのようなものがありますか？

その中でも、**数学的証明**はどのような点で優れていますか？

計算複雑性

コンピュータ上での計算にも、**難しさ（複雑性）**がある



<時間計算量>

計算複雑性 := 計算にかかる理論的リソース

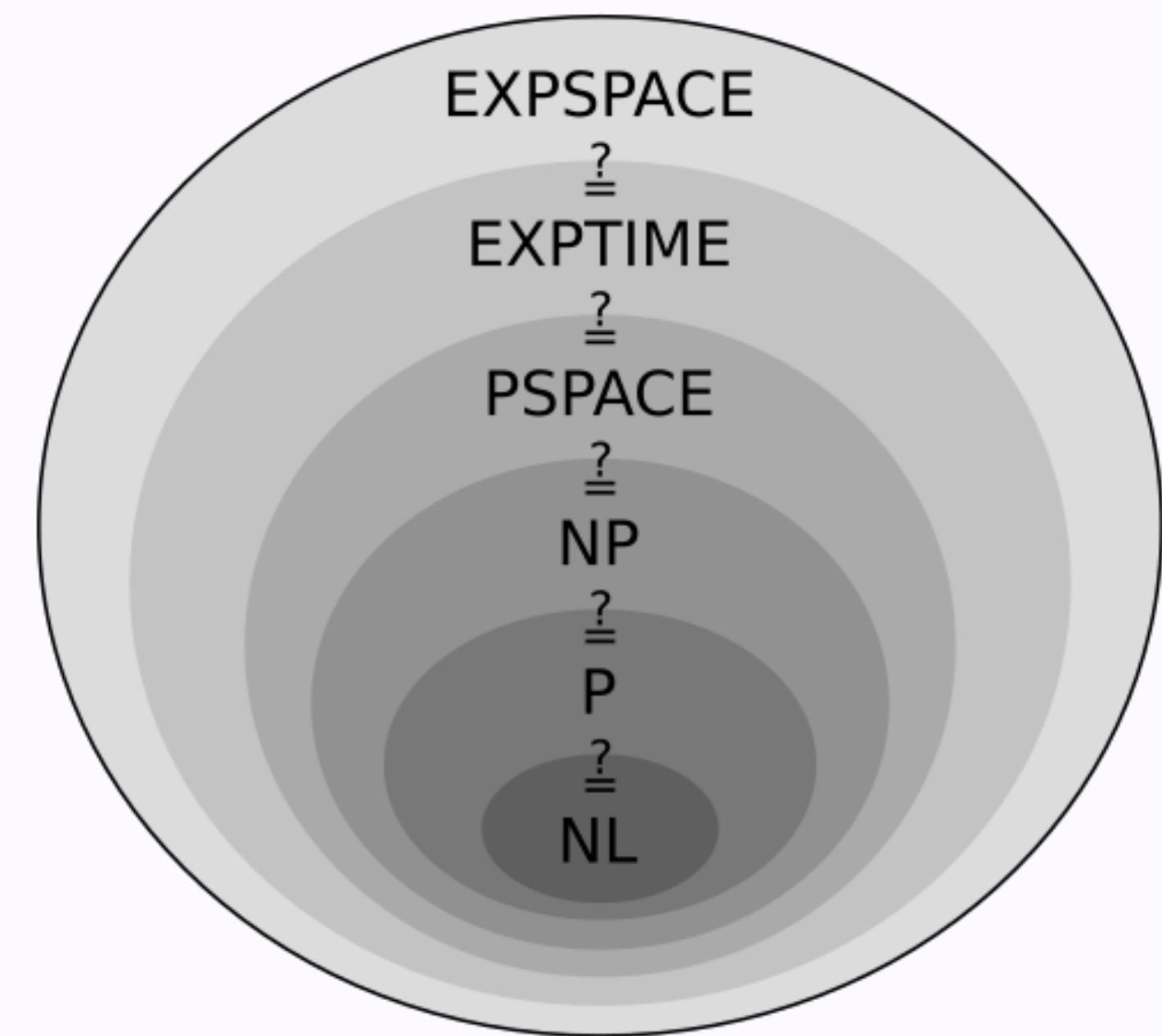
- 時間複雑性・計算量 (Time Complexity)
 - 入力サイズに対してどれだけのステップ数がかかるか
- 空間複雑性・計算量 (Space Complexity)
 - 入力サイズに対してどれだけのメモリー領域が必要か
- その他の複雑性
 - 通信複雑性 (Communication Complexity)
 - 回路複雑性 (Circuit Complexity)

計算複雑性クラス

コンピューターにとって簡単な問題とは、**多項式時間 (Polynomial Time)** で計算できるもの

計算複雑性クラス := 複雑性による計算問題の分類

- P (Polynomial Time)
 - 決定論的なコンピューターで多項式時間以下で解くことができる問題群
- NP (Non-Deterministic Polynomial Time)
 - 非決定論的な理論的コンピューターで多項式時間以下で解くことができる (とされる) 問題群
 - 決定論的なコンピューターで多項式時間以下で**検証することができる**問題群



<計算複雑性クラス>

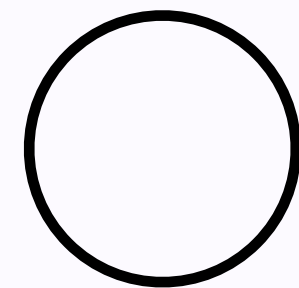
計算複雑性クラス

計算複雑性クラスによって、問題群同士の数学的（集合的）関係性が明らかになる

- **NP = PCP (Probabilistically Checkable Proofs)**
 - 効率的に検証可能な数学的証明を持つ問題群 (NP) は、検証者がその一部分をランダムにチェックするだけで検証可能な証明を持つ問題群 (PCP) に変換できる
- **IP (Interactive Proofs) = PSPACE (Polynomial Space)**
 - 無制限の計算資源を持つ証明者と、確率的に多項式時間で検証を行う検証者との間のやり取りで表現される問題群 (IP) は、多項式空間で解ける問題群 ($\text{PSPACE} \supseteq \text{NP}$) に等しい
- **MIP (Multi-Prover Interactive Proofs) = NEXPTIME (Non-Deterministic Exponential Time)**
 - IPにおいて、複数の独立した証明者を許容した問題群 (MIP) は、IPよりもさらに大きい複雑性クラスである問題群 (NEXPTIME) を解くことができる

計算複雑性クラスと証明システム

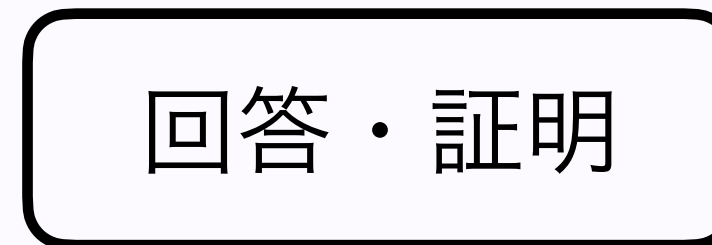
NP = 最も標準的な証明システムと捉えられる



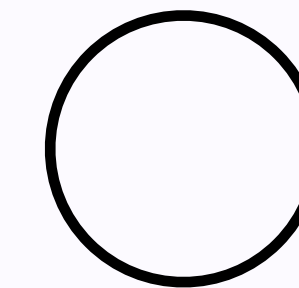
証明



回答・証明



検証

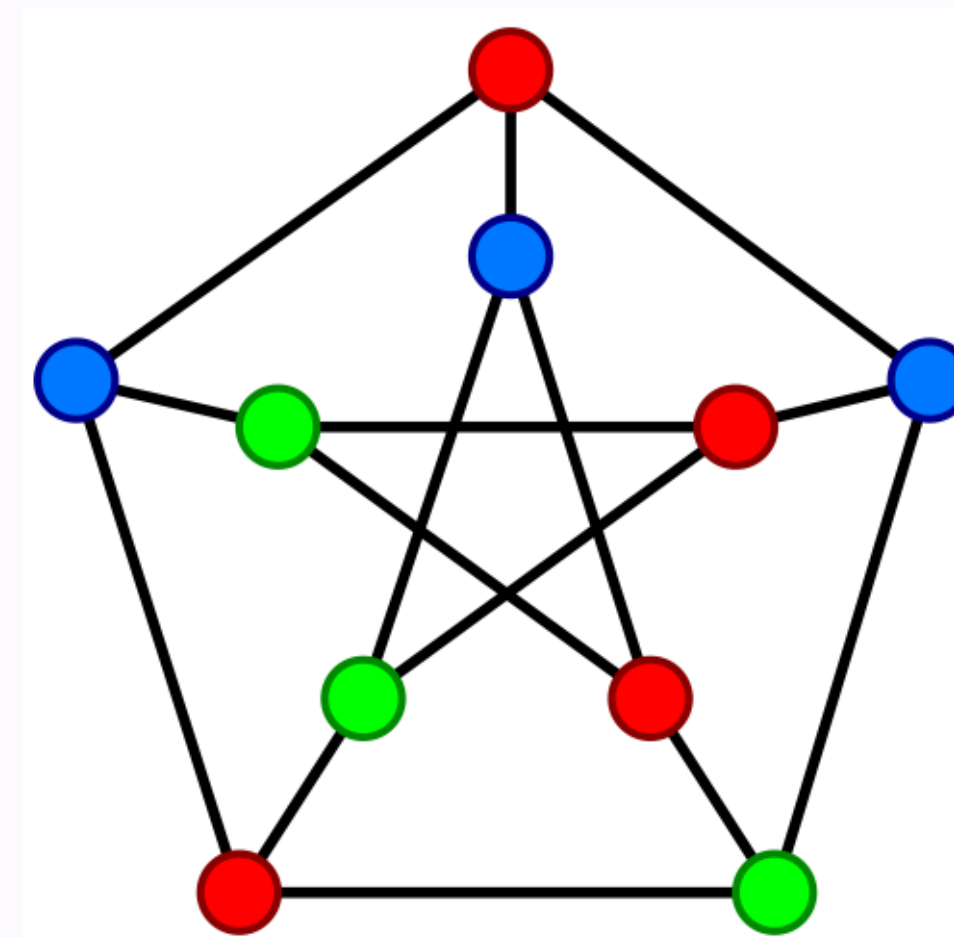


証明者 (Prover)

- 無限の計算能力 (i.e., 非決定論的コンピューター) を使って、問題を解く = 証明を見つけ出す

検証者 (Verifier)

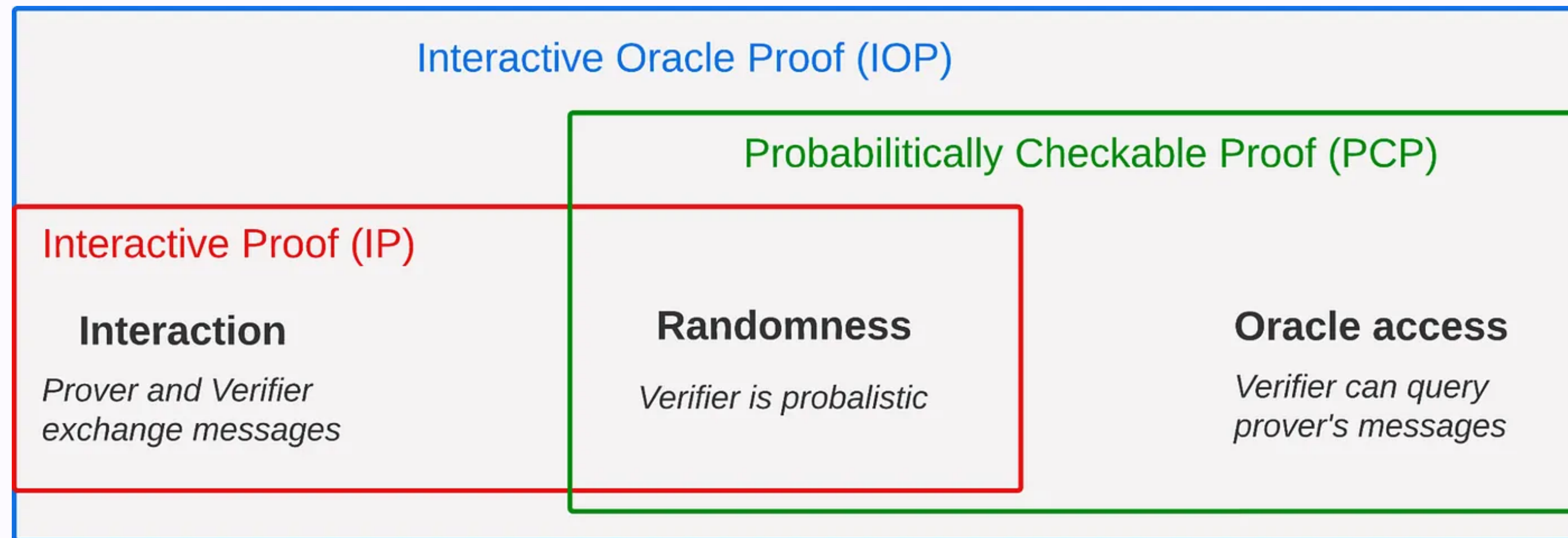
- **有限の計算能力** (i.e., 決定論的コンピューター) を使って、**多項式時間内で検証する**



< グラフ塗り分け問題 >

計算複雑性クラスと証明システム

NPにいろいろな条件を加えていくと、**現代の証明システム**になる



<IP · PCP · IOP>

証明の性質

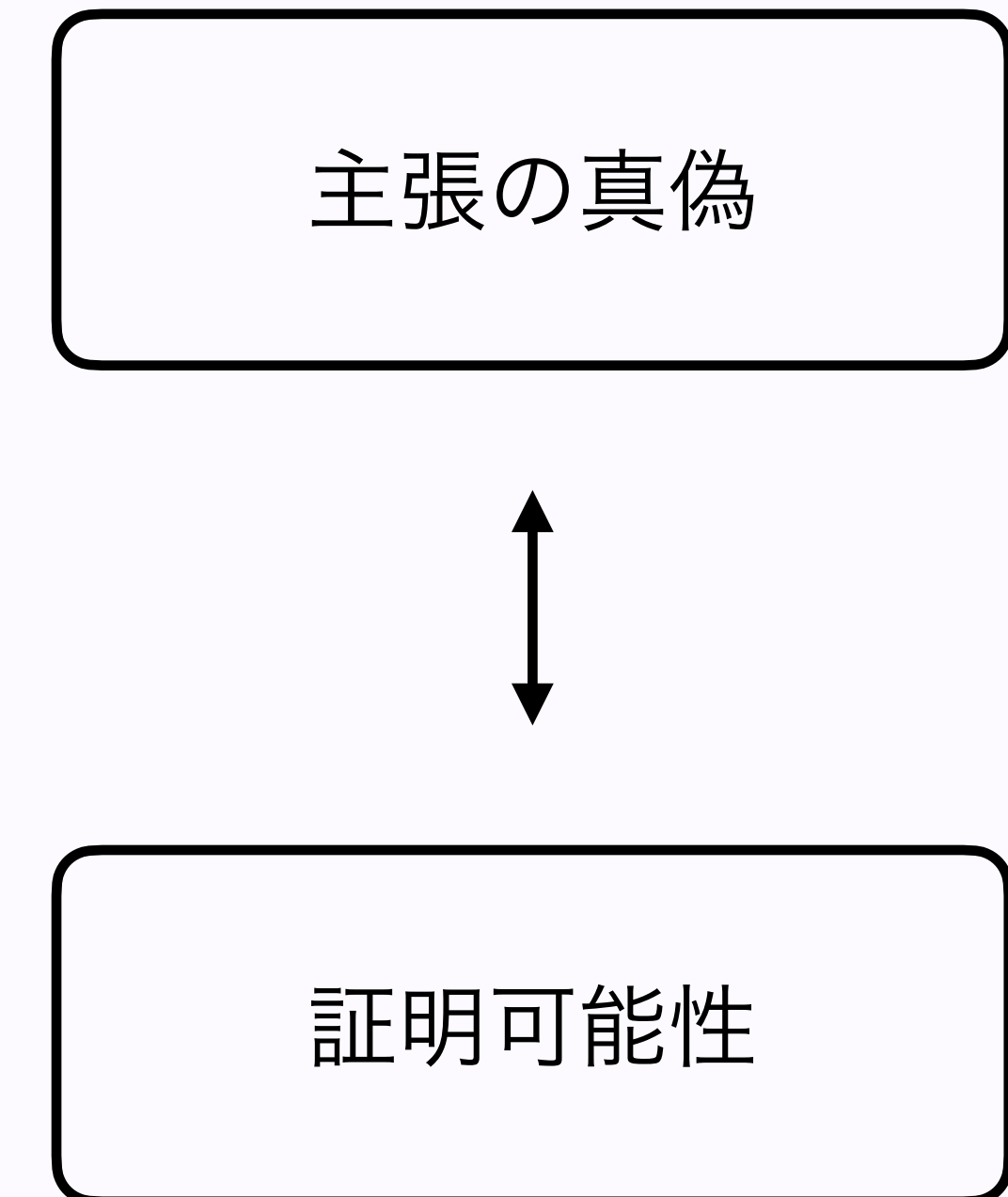
証明システムにおける証明には、満たすべき性質がある

証明システムの信頼性観点

- 完全性 (Completeness)
 - 主張が真であれば、常に証明可能である (i.e., 証明不可能 $\rightarrow$ 偽)
- 健全性 (Soundness)
 - 証明可能であれば、主張は真である (i.e., 偽 $\rightarrow$ 証明不可能)

証明システムのスケーラビリティ観点

- リソースの最適化 (e.g., 証明生成・検証コスト、証明サイズ、鍵サイズ)
 - $\rightarrow$ Q: 証明者と検証者向けの最適化ではどちらが重要？
 - 他に満たすべき性質はある？



目次

レクチャー

1. ゼロ知識証明の全体像 (Recap)
2. 証明とはなにか
3. **さまざまな証明スキーム**
4. ゼロ知識性の実現手法
5. まとめ

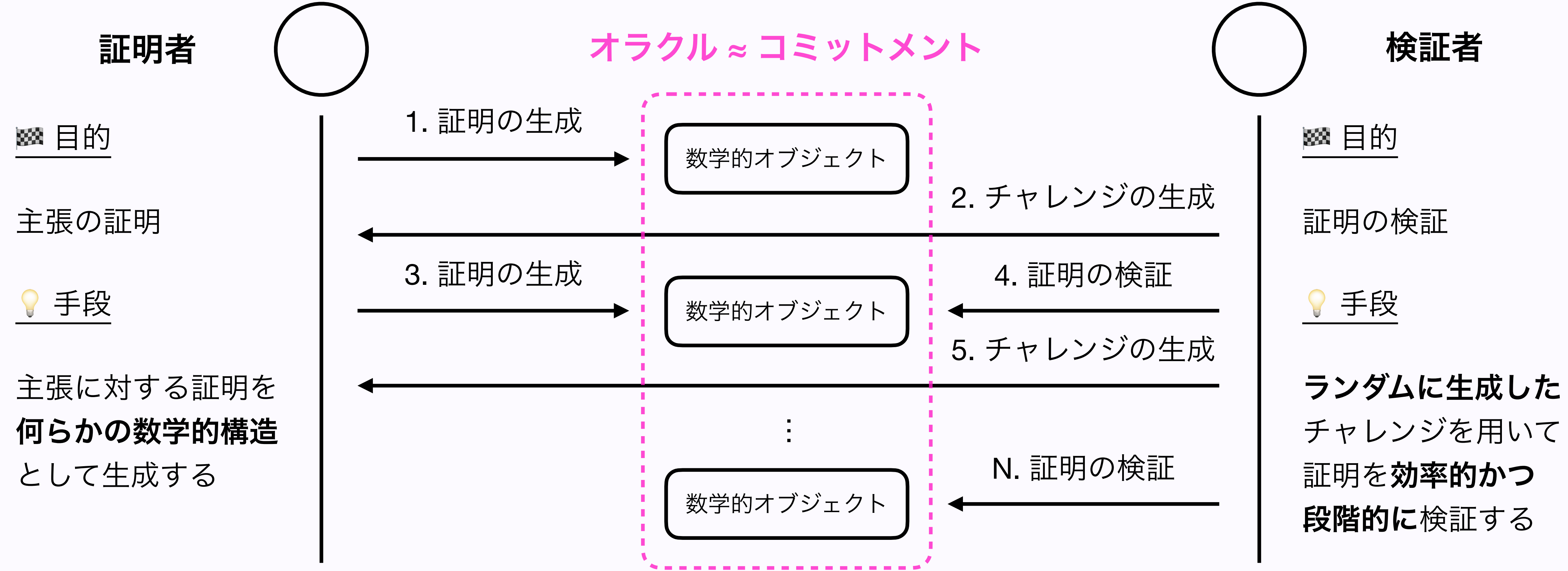
ワークショップ

～ZK, or not ZK～

ケーススタディ: AIエージェント保険

IOP (Interactive Oracle Proof)

現代のゼロ知識証明における証明スキームの主流はIOP



GKR (2008)

IPベースのスキーム: GKR (Goldwasser-Kalai-Rothblum)

A. 算術化方式

多重線形多項式
(Multilinear Polynomial)

1. 証明したい主張（計算）を算術回路に変換
2. 回路構造をレイヤー毎に多重線形多項式として表現

B. コミットメントスキーム

なし

オリジナルのGKRプロトコルでは
コミットメントスキームは不使用
→ ただし、組み合わせることで
効率化・ゼロ知識化ができる

C. 証明システム

SumCheckプロトコル

3. レイヤー毎にSum-Check
プロトコルを適用して検証
4. ランダム線型結合によって
検証における計算量を削減

GKRの仕組み

1. 計算の算術回路化

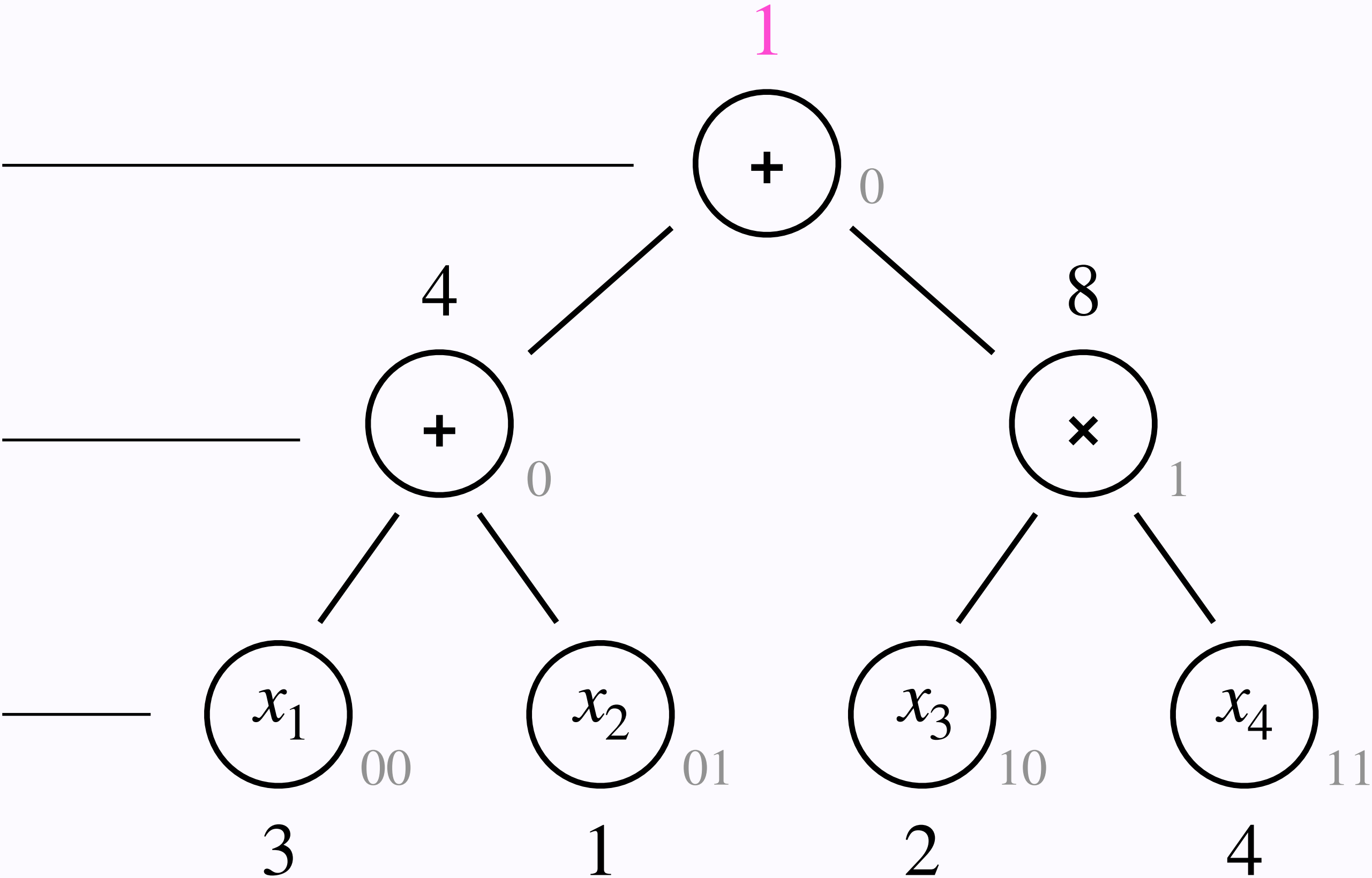
レイヤー毎に数式を分割する

上のレイヤーから順に証明

$$L_0 : y = y_0 + y_1$$

$$L_1 : \begin{cases} y_0 = x_1 + x_2 \\ y_1 = x_3 x_4 \end{cases}$$

$$L_2 : \{x_1, x_2, x_3, x_4\} \in \mathbb{F}_{11}$$



3. さまざまな証明スキーム

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2) + (x_3 x_4) \in \mathbb{F}_{11}[x]$$

GKRの仕組み

2. 計算結果とゲート構造のMLE (Multilinear Extension・多重線形拡張)

A. レイヤー毎の計算結果に関するMLE

$$L_0 : \tilde{W}_0(\phi) = 1$$

$$\begin{aligned} L_1 : \tilde{W}_1(z) &= 4(1 - z) + 8z \\ &= 4 + 4z \end{aligned}$$

$$\begin{aligned} L_2 : \tilde{W}_2(z_1, z_2) &= 3(1 - z_1)(1 - z_2) \\ &\quad + 1(1 - z_1)z_2 \\ &\quad + 2z_1(1 - z_2) \\ &\quad + 4z_1z_2 \\ &= 3 - z_1 - 2z_2 + 4z_1z_2 \end{aligned}$$

B. レイヤー毎のゲート構造に関するMLE

$$L_0 : \tilde{W}_0(\phi) = \sum_{a,b \in \{0,1\}} g_0(a, b)$$

$$g_0(a, b) = (1 - a)b(\tilde{W}_1(a) + \tilde{W}_1(b))$$

$$L_1 : \tilde{W}_1(z) = \sum_{a_1, a_2, b_1, b_2 \in \{0,1\}} g_1(a_1, a_2, b_1, b_2)$$

$$\begin{aligned} g_1(a_1, a_2, b_1, b_2) &= (1 - a_1)(1 - a_2)(1 - b_1)b_2(\tilde{W}_2(a_1, a_2) + \tilde{W}_2(b_1, b_2)) \\ &\quad + a_1(1 - a_2)b_1b_2(\tilde{W}_2(a_1, a_2)\tilde{W}_2(b_1, b_2)) \end{aligned}$$

GKRの仕組み

3. SumCheckプロトコル（1990）の適用

例) レイヤー#0へのSumCheckの適用: $\sum_{a,b \in \{0,1\}} g_0(a,b) = ? \quad \tilde{W}_0(\phi) = 1 \Leftrightarrow g_0(2,3) = ? \quad 4$

ラウンド#0

ラウンド#1

3-1. 一変数多項式化

$$\begin{aligned} p_1(x) &= \sum_{b \in \{0,1\}} g_0(x,b) \\ &= 12 - 8x - 4x^2 \\ &\equiv 1 + 3x + 7x^2 \pmod{11} \end{aligned}$$

$$\begin{aligned} p_2(x) &= \sum_{b \in \{0,1\}} g_1(2,y) \\ &= -5y - 4x^2 \\ &\equiv 6y + 7y^2 \pmod{11} \end{aligned}$$

3-2. 多項式の有効性検証

$$p_1(0) + p_1(1) = ? \quad \tilde{W}_0(\phi) = 1$$

$$p_2(0) + p_2(1) = ? \quad p_1(2) = 2$$

3-3. ランダム値での評価

$$p_1(r_1) = p_1(2) = 35 \equiv 2 \pmod{11}$$

$$p_2(r_2) = p_2(3) = 81 \equiv 4 \pmod{11}$$

4. 線形削減 (Line Reduction) による検証の効率化

例) レイヤー#0における複数の多項式評価を1つにまとめる: $g_0(2,3) = (1 - 2)3(\tilde{W}_1(2) + \tilde{W}_1(3))$

4-1. (2, 3)の2点を通る直線を定義

$$\begin{aligned} l(t) &= (1 - t)u + tv \\ &= (1 - t)2 + 3t \\ &= 2 + t \end{aligned}$$

4-2. $\tilde{W}_1$ を定義した直線に制限

$$\begin{aligned} q(t) &= \tilde{W}_1(l(t)) \\ &= 4 + 4(2 + t) \\ &\equiv 1 + 4t \pmod{11} \end{aligned}$$

$$\begin{cases} q(0) = 1 = \tilde{W}_1(2) \\ q(1) = 5 = \tilde{W}_1(3) \end{cases}$$

4-3. ランダム値での評価

$$\begin{aligned} q(r) &= q(4) \\ &= 1 + 16 \\ &\equiv 6 \pmod{11} \end{aligned}$$

STARK (2018)

IOPベースのスキーム①: STARK (Scalable Transparent ARgument of Knowledge)

A. 算術化方式

AIR

1. 証明したい主張（計算）を実行トレースに変換
2. 実行トレースに関する制約を多項式として表現（補間）

B. コミットメントスキーム

**マークルツリーベースの
多項式CS**

3. マークルツリーを用いてデータの堅牢性・証明の効率性（+ ゼロ知識性）を実現

C. 証明システム

FRIプロトコル

4. 多項式の次元数に関する性質を応用して、多項式の低次元性の証明に変換

STARKの仕組み

1. 計算の実行トレース化

ステップ数 ↓	加算結果 ↓	乗算結果 ↓	最終結果 ↓
t	a	m	y
0	0	0	0
1	4	0	0
2	4	8	0
3	4	8	12

実行トレース (execution trace)

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2) + (x_3 x_4) \in \mathbb{F}_{17}[x]$$

$$f(3, 1, 2, 4) = 12 \pmod{17}$$

- ← 初期状態
- ← $x_1 + x_2 = 3 + 1 = 4$
- ← $x_3 x_4 = 2 \cdot 4 = 8$
- ← $a + m = 4 + 8 = 12$

STARKの仕組み

2. 実行トレース制約の多項式化 (AIR)

評価ドメイン

遷移多項式 (trace polynomials)

	↓	↓	↓	↓
t	x	a	m	y
0	$\omega^0 = 1$	0	0	0
1	$\omega^1 = 4$	4	0	0
2	$\omega^2 = 16$	4	8	0
3	$\omega^3 \equiv 13$	4	8	12

制約多項式 (constraint polynomial)

実行トレース: $\omega = 4 \in \mathbb{F}_{17}$

2-1. ラグランジュ補間による遷移多項式化

$$A(X) = 3 - X - X^2 - X^3$$

$$M(X) = 4 + 6X + 7X^3$$

$$Y(X) = 3 + 12X + 14X^2 + 5X^3$$

2-2. 制約多項式として遷移多項式をまとめる

$$\begin{aligned} C(X) &= S_0(X)(A(\omega X) - 4) \\ &\quad + S_1(X)(M(\omega X) - 8) \\ &\quad + S_2(X)(Y(\omega X) - A(X) - M(X)) \\ &= 2 - 5X - 5X^2 - 2X^4 + 5X^5 + 5X^6 \\ &= 0 \end{aligned}$$

STARKの仕組み

3. マークルツリーを用いた多項式コミットメント

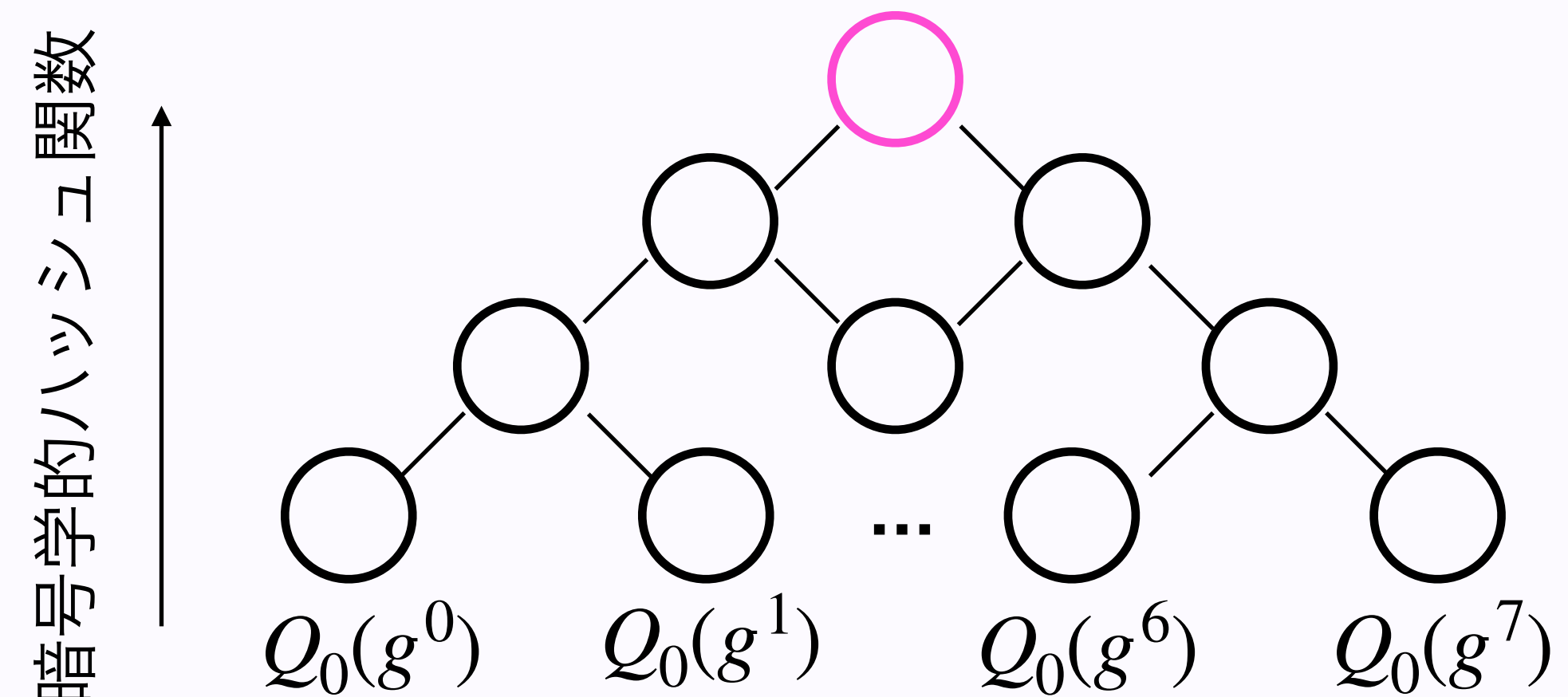
3-1. 制約多項式を商多項式の形に変換

因数定理 (Factor Theorem) から、制約多項式 $C(X)$ が評価ドメイン H の全ての点において $C(x_i) = 0$ となることは、 $C(X)$ が H の全ての点を根 (ルート) に持つような**消滅多項式 (Vanishing Polynomial)** $Z_H(X)$ で割り切れることと同じ

$$\begin{aligned} Q_0(X) &= \frac{C(X)}{Z_H(X)} = \frac{C(X)}{\prod_{v \in H} (X - v)} = \frac{C(X)}{X^n - 1} \\ &= 5X^2 + 5X - 2 \end{aligned}$$

3-2. 商多項式の評価値をコミットする

評価ドメインを元の数倍 (e.g., 4倍) 程度に拡張し、それぞれの点で商多項式 $Q_0(X)$ を評価した値をマークルツリーのリーフノードとしてコミットして、マークルルートを検証者に渡す



STARKの仕組み

Q. 各種計算量、通信量、証明サイズは？

4. 商多項式の低次元性証明 (FRI)

4-1. コミットフェーズ (by 証明者)

証明者は、商多項式を次数の偶奇で分割して表現し、
変数の置き換え・評価ドメインの半減・線形結合
によって商多項式の次数を徐々に削減していく

$$\begin{aligned} Q_0(X) &= Q_{0,even}(X^2) + XQ_{0,odd}(X^2) \\ &= (5X^2 - 2) + X \cdot 5 \\ &\downarrow \\ \begin{cases} Y = X^2 \\ \beta_1 = 3 \end{cases} \quad Q_1(Y) &= Q_{0,even}(Y) + \beta_1 Q_{0,odd}(Y) \\ &= (5Y - 2) + 3 \cdot 5 \\ &= 5Y + 13 \end{aligned}$$

4-2. クエリーフェーズ (by 検証者)

検証者は、リーフインデックスをランダムに選出、
証明者から値とそれぞれのマークルプルーフを取得し、
それらを用いて線形結合の正しさを効率的に検証する
(Schwartz-Zippelの補題により健全性は高い)

$$\begin{aligned} \begin{cases} Q_0(x) &= Q_{0,even}(x^2) + xQ_{0,odd}(x^2) \\ Q_0(-x) &= Q_{0,even}(x^2) - xQ_{0,odd}(x^2) \end{cases} \\ \Leftrightarrow \begin{cases} Q_{0,even}(x^2) &= \frac{Q_0(x) + Q_0(-x)}{2} \\ Q_{0,odd}(x^2) &= \frac{Q_0(x) - Q_0(-x)}{2x} \end{cases} \end{aligned}$$

PLONK (2019)

IOPベースのスキーム②: PLONK (Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge)

A. 算術化方式

Plonkish
(複数の一変数多項式)

1. 証明したい主張（計算）を算術回路に変換
2. 回路構造や回路の入出力値を一変数多項式として表現

B. コミットメントスキーム

**KZGベースの
多項式CS**

3. 多項式を楕円曲線上の一要素に変換（固定）し、効率的な証明（+ ゼロ知識性）を実現

C. 証明システム

**多項式IOP
(Polynomial IOP)**

4. KZGの評価証明に加えて、ランダムな評価ポイントによって多項式制約を効率的に検証

PLONKの仕組み

1. 計算の算術回路化

PLONKでは、算術回路をゲートに関して、
3つのパーツに分けてベクトル → 多項式化する

1. ウィットネス多項式 (Witness Polynomials)

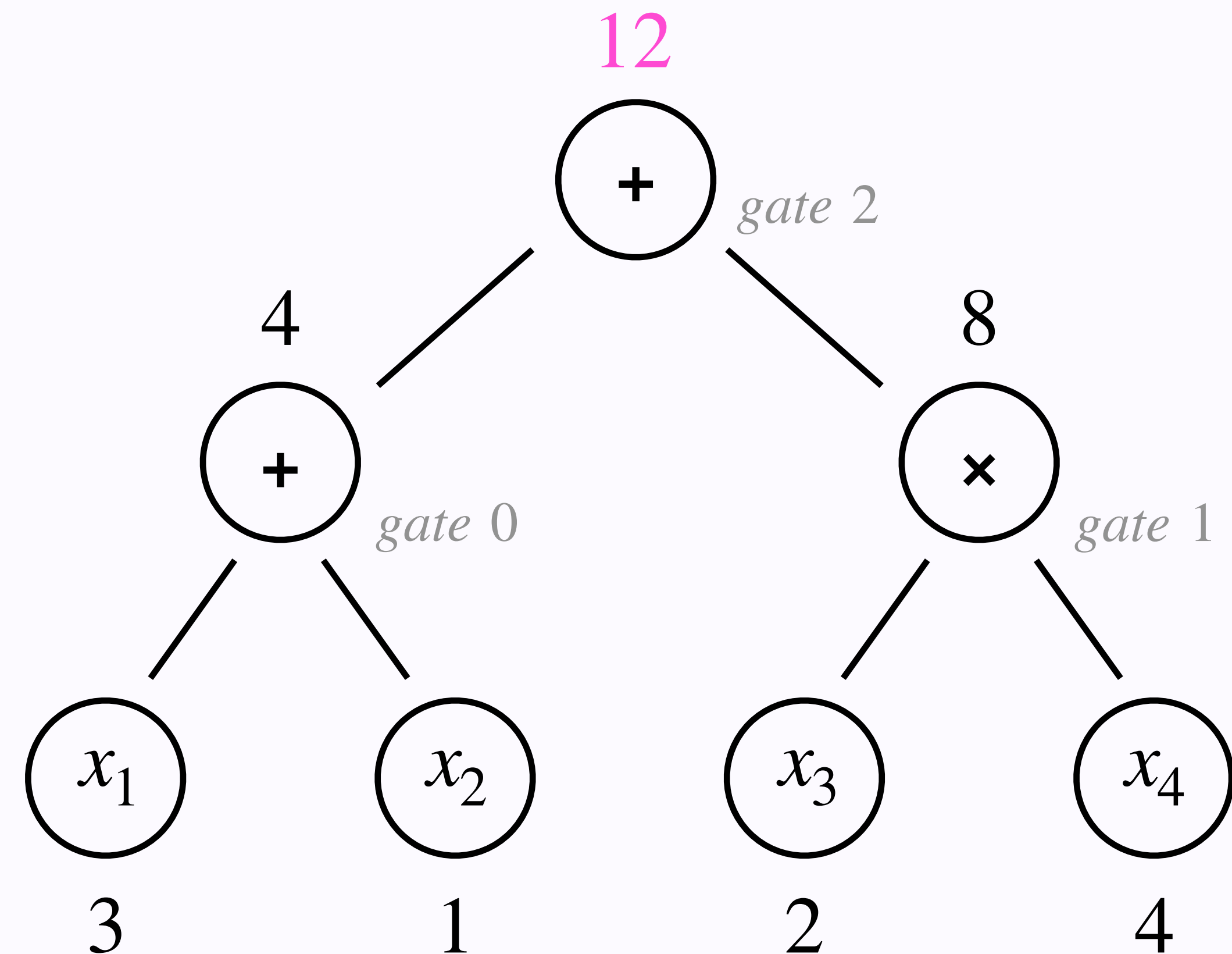
→ ゲート毎の入出力値に関する多項式

2. セレクター多項式 (Selector Polynomials)

→ ゲート毎の構造に関する多項式

3. ワイヤリング多項式 (Wiring Polynomials)

→ ゲート間の繋がりに関する多項式



3. さまざまな証明スキーム

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2) + (x_3 x_4) \in \mathbb{F}_{17}[x] \quad 26$$

PLONKの仕組み

2. 入力値と回路構造の一変数多項式化

11個のベクトルをそれぞれラグランジュ補間して多項式にする

ウィットネス多項式

	入力左 ↓	入力右 ↓	出力 ↓
	L	R	O
ゲート#0	3	1	4
ゲート#1	2	4	8
ゲート#2	4	8	12

セレクター多項式

入力左 係数 ↓	入力右 係数 ↓	乗算bit ↓	出力 係数 ↓	定数 ↓
Q_L	Q_R	Q_M	Q_O	Q_C
1	1	0	-1	0
0	0	1	-1	0
1	1	0	-1	0

ワイヤリング多項式

入力左 インデックス ↓	入力右 インデックス ↓	出力 インデックス ↓
σ_L	σ_R	σ_O
$\omega^0 k_0 = k_0$	$\omega^0 k_1 = k_1$	$\omega^2 k_0 = 16k_0$
$\omega^1 k_0 = 4k_0$	$\omega^1 k_1 = 4k_1$	$\omega^2 k_1 = 16k_1$
$\omega^0 k_2 = k_2$	$\omega^1 k_2 = 4k_2$	$\omega^2 k_2 = 16k_2$

3. さまざまな証明スキーム

PLONKの仕組み

3. KZGコミットメント

3-1. セットアップ (trusted setup)

コミットメントや検証ステップで必要となる

SRS (Structured Reference String) をMPC経由で算出

$$SRS = \{\tau g_1, \tau^2 g_1, \dots, \tau^d g_1, g_2, \tau g_2\}$$

3-3. 開示 (open)

ランダムに選出されたポイント $\alpha \in \mathbb{F}_p$ での多項式評価を
商多項式 $q(X)$ へのコミットメントに変換

$$q(X) := \frac{f(X) - f(\alpha)}{X - \alpha} \rightarrow Com_q := q(\tau)g_1$$

3-2. コミット (commit)

SRSを用いて、一変数多項式 $f(X) := \sum_{i=0}^d c_i X_i$
へコミット

$$Com_f := f(\tau)g_1$$

3-4. 検証 (verify)

ペアリングの双線型性 (bilinearity) を用いて
コミットメントを検証

$$\begin{aligned} f(\tau) &= ?(\tau - \alpha)q(\tau) + f(\alpha) \\ \Leftrightarrow Com_f &= ?(\tau - \alpha)Com_q + f(\alpha)g_1 \end{aligned}$$

PLONKの仕組み

Q. 各種計算量、通信量、証明サイズは？

4. 多項式制約の証明・検証

インプット制約: KZGの開示証明 (opening proofs) を利用するのみ
→ バッチ化による最適化も可能

ゲート制約: ウィットネス多項式とセクター多項式で全てのゲート制約を表現し、**ゼロテスト**を行う
→ (STARKと同様に) 消滅多項式 $Z_H(X)$ を用いて商多項式 $Q(X)$ への変換を行う

$$f(X) = Q_L(X)L(X) + Q_R(X)R(X) + Q_M(X)L(X)R(X) + Q_o(X)O(X) + Q_c(X) = ?0$$

コピー制約: ワイヤリング多項式によるインデックスの入れ替え前後 (それぞれ $f(X)$ と $g(X)$) で、ウィットネス多項式の評価値が並び替えになっていることを**大積 (grand product)** で示す
→ **累算器 (accumulator)** $p(X)$ によるゼロテスト問題、さらに商多項式への変換を行う

$$p(X) = \begin{cases} p(\omega^0) = 1 \\ p(\omega^{i+1}) = p(\omega^i) \frac{f(X)}{g(X)} \end{cases} \quad \Leftrightarrow \quad p'(X) = p(\omega X)g(X) - p(X)f(X) = 0$$

目次

レクチャー

1. ゼロ知識証明の全体像 (Recap)
2. 証明とはなにか
3. さまざまな証明スキーム
4. ゼロ知識性の実現手法
5. まとめ

ワークショップ

～ZK, or not ZK～

ケーススタディ: AIエージェント保険

証明スキームのゼロ知識化

STARKとPLONKのゼロ知識 (ZK) 化 $\equiv$ 多項式のマスキング

対象の多項式に対して、多項式の評価ドメイン (i.e., 1の原始累乗根 ω^i) 内では0となり、評価ドメイン外ではランダムな値となるような**ブラインド多項式 (blind polynomial)** を加える

$$\tilde{f}(X) := f(X) + Z_H(X)R(X) \quad \text{where} \quad Z_H(X) = \prod_{h \in H} (X - h)$$

STARK

対象: 遷移多項式、制約多項式

→ FRIも踏まえて十分なランダムネスを確保

→ Merkleコミットメントでもランダムソルトを追加

PLONK

対象: ウィットネス多項式、ワイヤリング多項式、各商多項式

Q. なぜセレクター多項式はブラインド化しない？

証明スキームのゼロ知識化

GKRのゼロ知識 (ZK) 化

多項式自体のマスキング

+

コミットメントスキームの利用

2. マスキング用の多変数多項式 $R(X)$ をコミット
→ マークルツリーが用いられることが多い
3. マスキングされた状態でSumCheckを実行
→ ランダムな評価ポイントが定まるまで
→ $f(X) + R(X)$
4. (SumCheck終盤) 引き算によって相殺
→ $V_{original} = V_{total} - V_{random}$

1. 回路構造を表した多変数多項式をコミット
→ インデックス毎にベクトル化してバッチ化も可能

Q. どちらかのみではなぜNG?

5. コミットメント開示証明の検証 (多項式評価の代替)
→ マークルパスの検証 + FRIの適用

目次

レクチャー

1. ゼロ知識証明の全体像 (Recap)
2. 証明とはなにか
3. さまざまな証明スキーム
4. ゼロ知識性の実現手法
5. まとめ

ワークショップ

～ZK, or not ZK～

ケーススタディ: AIエージェント保険

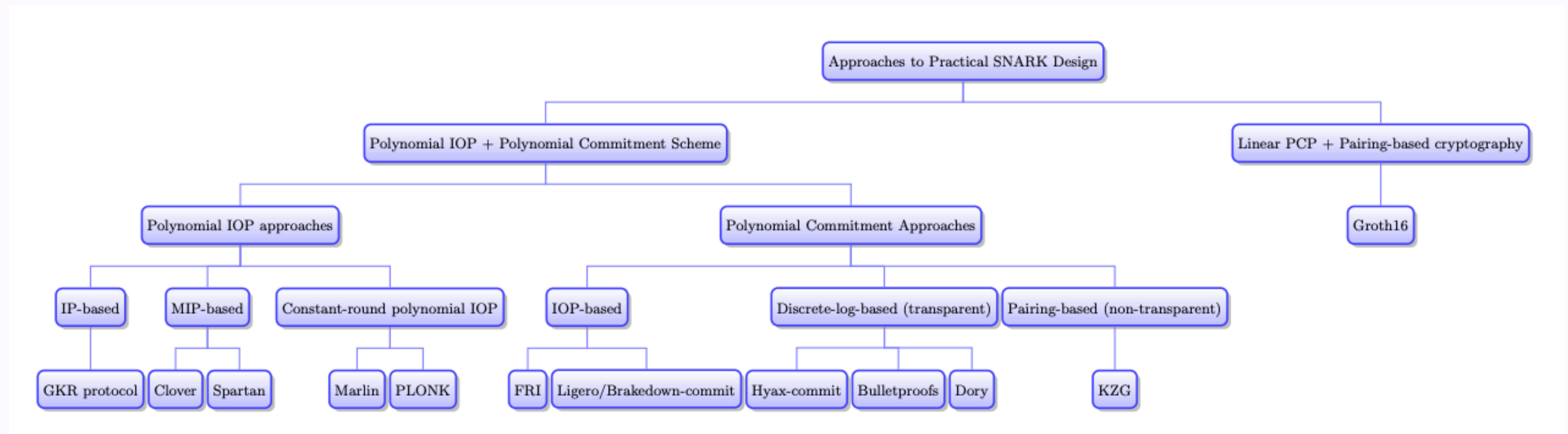
ゼロ知識証明まとめ

ゼロ知識証明 := 証明 + ゼロ知識性

- 社会や日々の生活において「証明」という概念は普遍的なもの
 - 現代の証明システム (e.g., IOP) は、効率的な検証を可能にする数学的証明の1つ
- ゼロ知識証明は「算術化」「コミットメントスキーム」「証明システム」の組み合わせ
 - 「完全性・健全性の高さ」「ゼロ知識性の強さ」によって、スキームの堅牢性が測れる
 - 「証明者と検証者の計算量」「証明サイズ」などによって、スキームの性能が測れる
- 同じゼロ知識スキームを指していても、論文や実装によって細部が異なる場合がある
 - GKR: Virgo/Libra (FRI/KZGの導入)、Jolt (部分的にGKRを適用)
 - STARK: Redshift (AIR → Plonkish)、zkSTARK (Deep FRI)
 - PLONK: Halo2 (KZG → IPA)、Plonky2/3 (Plonkish/AIR + FRI)

ゼロ知識証明まとめ

大枠と構成要素は共通、組み合わせはさまざま



< SNARKの分類 >

目次

レクチャー

1. ゼロ知識証明の全体像 (Recap)
2. 証明とはなにか
3. さまざまな証明スキーム
4. ゼロ知識性の実現手法
5. まとめ

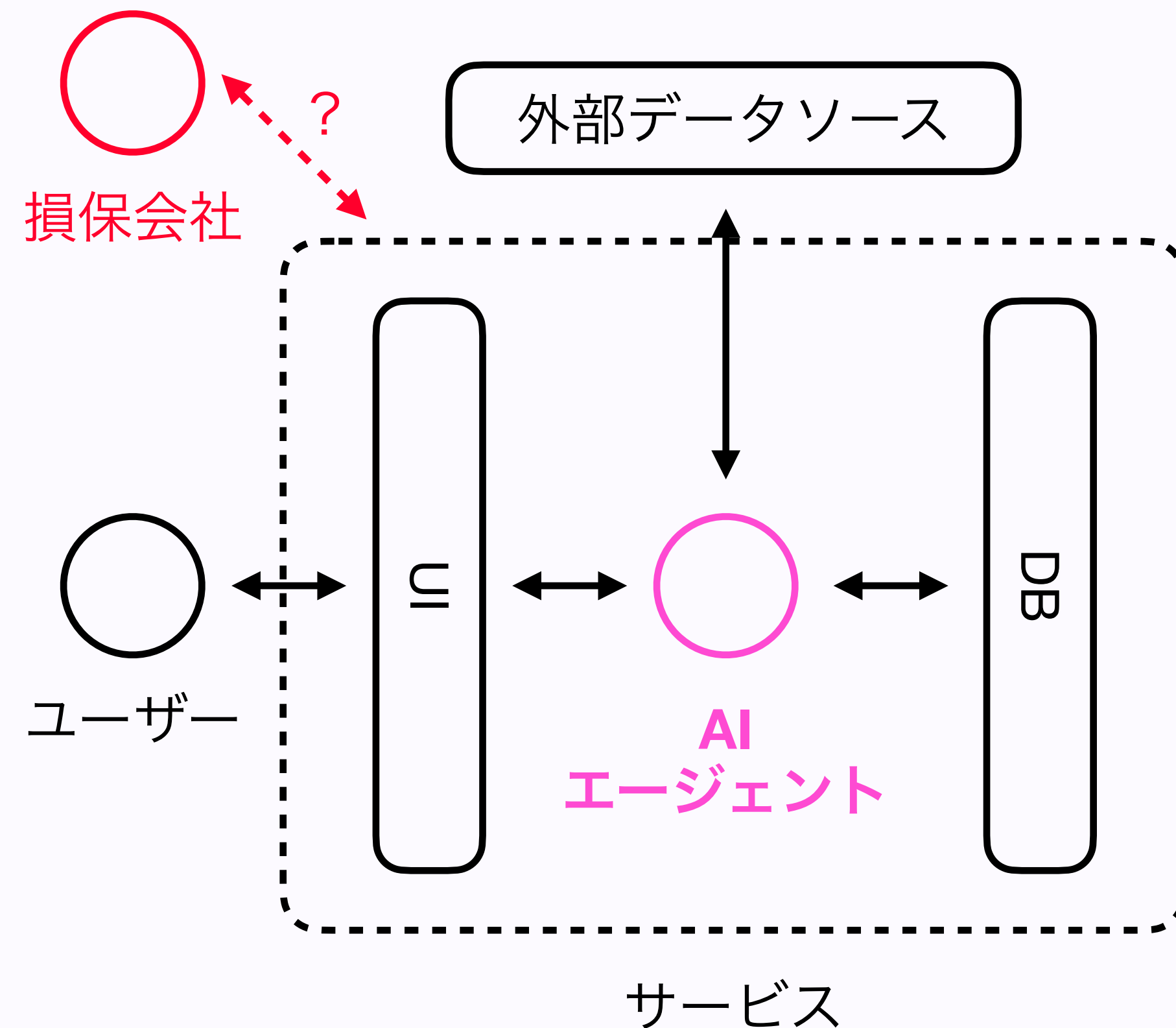
ワークショップ

～ZK, or not ZK～

ケーススタディ: AIエージェント保険

ワークショップ

ZK, or not ZK (ケーススタディ: AIエージェント保険)



グループ課題

AIエージェントの運用実績 (e.g., タスク完了率) を
サービスの機密データ (e.g., 顧客データ) を渡さずに
外部の保険会社に対して証明するシステムの設計

検討すべき事項

- 指標 (e.g., ISO) と運用実績の測定・証明方法
- 機密データの秘匿方法・具体的スキーム
 - トラストモデル (誰・何を信頼するモデルか)
 - 現実的なトレードオフ